

Atividade Orientada

João Gabriel Miguêz da Silva - 2023008890

Felipe Bastos Vargas - 2023008881

Exercício 1

In [15]:

```
import math
import numpy as np
import itertools

espaco_amostral = []

for i in range(1,5):
    for j in range(1,5):
        espaco_amostral.append((i, j))

print("Espaco amostral:")
print(espaco_amostral)

valores_X = []
contagem_frequencia = [0] * 9

for i, j in espaco_amostral:
    soma = i + j
    valores_X.append(soma)
    contagem_frequencia[soma] += 1

valores_Xi = contagem_frequencia[5]/len(espaco_amostral)

print("\nValores de X = soma:")
print(valores_X)

print("\nProbabilidade de X = 5:")
print(f"{valores_Xi*100}%")

for k in range(0,9):
    valores_Xi = contagem_frequencia[k]/len(espaco_amostral)
    print(f"\nProbabilidade de X = {k}:")
    print(f"{valores_Xi*100}%")
```

Espaco amostral:

[(1, 1), (1, 2), (1, 3), (1, 4), (2, 1), (2, 2), (2, 3), (2, 4), (3, 1), (3, 2), (3, 3), (3, 4), (4, 1), (4, 2), (4, 3), (4, 4)]

Valores de X = soma:

[2, 3, 4, 5, 3, 4, 5, 6, 4, 5, 6, 7, 5, 6, 7, 8]

Probabilidade de X = 5:

25.0%

Probabilidade de X = 0:

0.0%

Probabilidade de $X = 1$:

0.0%

Probabilidade de $X = 2$:

6.25%

Probabilidade de $X = 3$:

12.5%

Probabilidade de $X = 4$:

18.75%

Probabilidade de $X = 5$:

25.0%

Probabilidade de $X = 6$:

18.75%

Probabilidade de $X = 7$:

12.5%

Probabilidade de $X = 8$:

6.25%

Exercício 2

In [16]:

```
espaco_amostral = []

# 0 = cara
# 1 = coroa
for i in range(0,2):
    for j in range(0,2):
        for k in range(0,2):
            for l in range(0,2):
                for m in range(0,2):
                    espaco_amostral.append((i,j,k,l,m))
print(f"Espaço amostral: {espaco_amostral}")

valores_X = [] #soma

for i,j,k,l,m in espaco_amostral:
    valores_X.append(i+j+k+l+m)

total_eventos = len(espaco_amostral)

# (a)  $P(X > 3) = P(X = 4) + P(X = 5)$ 
contagem_X4 = valores_X.count(4)
contagem_X5 = valores_X.count(5)
P_X_maior_3 = (contagem_X4 + contagem_X5) / total_eventos

# (b)  $P(X = 2)$ 
contagem_X2 = valores_X.count(2)
P_X_igual_2 = contagem_X2 / total_eventos
```

```
# (c)  $E(X)$  = valor esperado
E_X = sum(valores_X) / total_eventos

# Resultados
print(f"Total de eventos: {total_eventos}")
print(f"\n(a)  $P(X > 3) = \{P\_X\_maior\_3\}$ ")
print(f"      ({contagem_X4} eventos com X=4 + {contagem_X5} eventos com X=5) / {total_eve
print(f"      = {P_X_maior_3:.4f} ou {P_X_maior_3 * 100:.1f}%")

print(f"\n(b)  $P(X = 2) = \{P\_X\_igual\_2\}$ ")
print(f"      ({contagem_X2} eventos) / {total_eventos}")
print(f"      = {P_X_igual_2:.4f} ou {P_X_igual_2 * 100:.1f}%")

print(f"\n(c)  $E(X) = \{E\_X\}$ ")
print(f"      Soma total de caras: {sum(valores_X)} / {total_eventos}")
print(f"      = {E_X:.2f} caras em média")
```

Espaço amostral: [(0, 0, 0, 0, 0), (0, 0, 0, 0, 1), (0, 0, 0, 1, 0), (0, 0, 0, 1, 1), (0, 0, 1, 0, 0), (0, 0, 1, 0, 1), (0, 0, 1, 1, 0), (0, 0, 1, 1, 1), (0, 1, 0, 0, 0), (0, 1, 0, 0, 1), (0, 1, 0, 1, 0), (0, 1, 0, 1, 1), (0, 1, 1, 0, 0), (0, 1, 1, 0, 1), (0, 1, 1, 1, 0), (0, 1, 1, 1, 1), (1, 0, 0, 0, 0), (1, 0, 0, 0, 1), (1, 0, 0, 1, 0), (1, 0, 0, 1, 1), (1, 0, 1, 0, 0), (1, 0, 1, 0, 1), (1, 0, 1, 1, 0), (1, 0, 1, 1, 1), (1, 1, 0, 0, 0), (1, 1, 0, 0, 1), (1, 1, 0, 1, 0), (1, 1, 0, 1, 1), (1, 1, 1, 0, 0), (1, 1, 1, 0, 1), (1, 1, 1, 1, 0), (1, 1, 1, 1, 1)]

Total de eventos: 32

(a) $P(X > 3) = 0.1875$
 (5 eventos com X=4 + 1 eventos com X=5) / 32
 = 0.1875 ou 18.8%

(b) $P(X = 2) = 0.3125$
 (10 eventos) / 32
 = 0.3125 ou 31.2%

(c) $E(X) = 2.5$
 Soma total de caras: 80 / 32
 = 2.50 caras em média

Exercício 3

In [17]:

```
# Representação das bolas: 0,1,2 são Vermelhas | 3,4 são Azuis
urnas = [0, 1, 2, 3, 4]
espaco_amostral = []

for i in urnas:
    for j in urnas:
        if i != j:
            espaco_amostral.append((i, j))

print("Espaco amostral:")
print(espaco_amostral)

valores_X = []
for i, j in espaco_amostral:
    contagem_vermelhas = 0
    if i < 3: contagem_vermelhas += 1
```

```

    if j < 3: contagem_vermelhas += 1
    valores_X.append(contagem_vermelhas)

print("\nValores de X = soma:")
print(valores_X)

contagem_frequencia = [0] * 3

for x in valores_X:
    contagem_frequencia[x] += 1

print("\nDistribuição de Probabilidades de X:")
probabilidades = []

for k in range(3):
    prob = contagem_frequencia[k] / len(espaco_amostral)
    probabilidades.append(prob)
    print(f"P(X = {k}): {prob*100:.1f}%")

valores_possiveis = [0, 1, 2]
esperanca = np.average(valores_possiveis, weights=probabilidades)

print(f"\nEsperanca: {esperanca:.2f}")

```

Espaco amostral:

```
[(0, 1), (0, 2), (0, 3), (0, 4), (1, 0), (1, 2), (1, 3), (1, 4), (2, 0), (2, 1), (2, 3),
(2, 4), (3, 0), (3, 1), (3, 2), (3, 4), (4, 0), (4, 1), (4, 2), (4, 3)]
```

Valores de X = soma:

```
[2, 2, 1, 1, 2, 2, 1, 1, 2, 2, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0]
```

Distribuição de Probabilidades de X:

P(X = 0): 10.0%

P(X = 1): 60.0%

P(X = 2): 30.0%

Esperanca: 1.20

Exercício 4

In [18]:

```

espaco_amostral = []

for i in range (0,5):
    for j in range (0,5):
        espaco_amostral.append((i,j))

valores_X = []    # soma
valores_Y = []    # maximo

for i, j in espaco_amostral:
    valores_X.append(i + j)
    valores_Y.append(max(i, j))

print("\nValores de X = soma:")
print(valores_X)

```

```

print("\nValores de Y = maximo:")
print(valores_Y)

valores_possiveis_X = sorted(set(valores_X))
valores_possiveis_Y = sorted(set(valores_Y))

print("\nValores possiveis de X = soma:")
print(valores_possiveis_X)

print("\nValores possiveis de Y = maximo:")
print(valores_possiveis_Y)

frequencias = {}

for valor in valores_X:
    if valor in frequencias:
        frequencias[valor] += 1
    else:
        frequencias[valor] = 1

print("\nFrequencias observadas:")
print(frequencias)

```

Valores de X = soma:

[0, 1, 2, 3, 4, 1, 2, 3, 4, 5, 2, 3, 4, 5, 6, 3, 4, 5, 6, 7, 4, 5, 6, 7, 8]

Valores de Y = maximo:

[0, 1, 2, 3, 4, 1, 1, 2, 3, 4, 2, 2, 2, 3, 4, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4]

Valores possiveis de X = soma:

[0, 1, 2, 3, 4, 5, 6, 7, 8]

Valores possiveis de Y = maximo:

[0, 1, 2, 3, 4]

Frequencias observadas:

{0: 1, 1: 2, 2: 3, 3: 4, 4: 5, 5: 4, 6: 3, 7: 2, 8: 1}

Exercício 5

Neste exercício 5, usamos o valor de $n = 5$ para teste, mas qualquer valor poderia ser usado.

Então, calculamos todas as permutações possíveis, para podermos determinar o espaço amostral e todas as situações em que uma carta se mantém no mesmo lugar. Depois disso, podemos calcular a probabilidade da quantidade de cada carta estar ordenada, e a esperança matemática.

O que é interessante desse problema, é que independente do número de cartas que sejam embaralhadas, a esperança sempre será 1.

In [19]:

```

import numpy as np

n = 7

cartoes_originais = list(range(1, n + 1))

```

```

espaco_amostral = list(itertools.permutations(cartoes_originais))

print(f"\nNúmero total de permutações: {len(espaco_amostral)}")

valores_X_lista = []

for permutacao in espaco_amostral:
    coincidencias = 0
    for i in range(n):
        if permutacao[i] == cartoes_originais[i]:
            coincidencias += 1
    valores_X_lista.append(coincidencias)

contagem_frequencia = [0] * (n + 1)
for x in valores_X_lista:
    contagem_frequencia[x] += 1

print("\ncontagem frequencia:")
print(contagem_frequencia)

probabilidades = []
for k in range(n + 1):
    prob = contagem_frequencia[k] / len(espaco_amostral)
    probabilidades.append(prob)

print("\nprobabilidades:")
print(probabilidades)

valores_possiveis = list(range(n + 1))
esperanca = np.average(valores_possiveis, weights=probabilidades)

print(f"\nEsperança Matemática E(X) para n={n}: {esperanca:.2f}")

```

Número total de permutações: 5040

contagem frequencia:
[1854, 1855, 924, 315, 70, 21, 0, 1]

probabilidades:
[0.3678571428571429, 0.3680555555555556, 0.18333333333333332, 0.0625, 0.013888888888888888, 0.004166666666666667, 0.0, 0.0001984126984126984]

Esperança Matemática E(X) para n=7: 1.00

Exercício 6

In [20]:

```

# Exercício 6
import math
import matplotlib.pyplot as plt
import numpy as np
# cara = p          #probabilidade de sucesso
# coroa = 1-p       #probabilidade de fracasso
# n lançamentos
# X = numero de cara obtidas

```

```

def P(k, n, p):
    if k < 0 or k > n:
        return 0

    coefBinomial = math.factorial(n) // (math.factorial(k) * math.factorial(n-k))
    probabilidade = coefBinomial * (p ** k) * ((1-p) ** (n-k))
    return probabilidade

def E(p,n):
    return p * n

def V(p,n):
    return n * p * (1-p)

p_sucesso = float(input("Entre com a probabilidade de sucesso (0.0 <= p <= 1.0): "))
n = int(input("Entre com o numero de lances: "))
k = int(input("Entre com o numero de sucessos para se calcular a probabilidade: "))

print(f"Probabilidade de Cara: {P(k, n, p_sucesso)}")
print(f"Esperança E(X): {E(p_sucesso,n)}")
print(f"Variância V(X): {V(p_sucesso, n):.2f}")

# 1. Gerar todos os valores possíveis de k (de 0 até n)
eixo_x = np.arange(0, n + 1)

# 2. Calcular a probabilidade para cada valor de k usando a função P
eixo_y = [P(i, n, p_sucesso) for i in eixo_x]

# 3. Criar o gráfico
plt.figure(figsize=(10, 6))
plt.bar(eixo_x, eixo_y, color='skyblue', edgecolor='navy', alpha=0.7)
plt.title(f'Distribuição Binomial (n={n}, p={p_sucesso})', fontsize=14)
plt.xlabel('Número de Caras (k)', fontsize=12)
plt.ylabel('Probabilidade P(X = k)', fontsize=12)
plt.xticks(eixo_x)
plt.grid(axis='y', linestyle='--', alpha=0.6)
for i in range(len(eixo_x)):
    plt.text(eixo_x[i], eixo_y[i], f'{eixo_y[i]:.3f}', ha='center', va='bottom', fontsize=10)

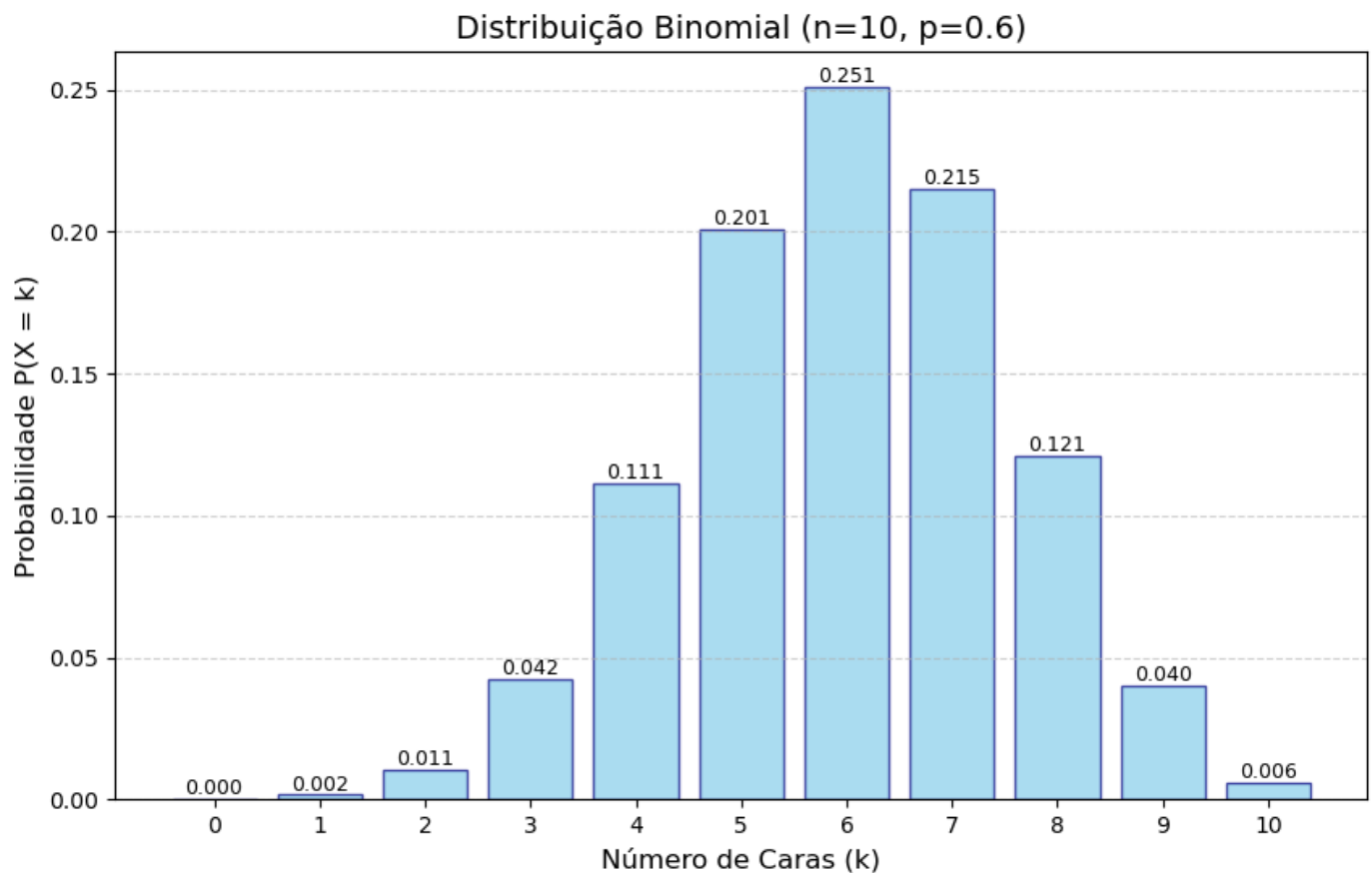
plt.show()

```

Probabilidade de Cara: 0.20065812480000003

Esperança E(X): 6.0

Variância V(X): 2.40



Exercício 7

In [21]:

```
x = np.array([0, 1, 2, 3]) #Dados da Tabela
prob = np.array([0.1, 0.3, 0.4, 0.2])

esperanca_x = np.average(x, weights=prob)
esperanca_x2 = np.average(x**2, weights=prob)
variancia = esperanca_x2 - (esperanca_x**2)

print(f"\nEsperanca: {esperanca_x:.2f}")
print(f"\nEsperanca^2: {esperanca_x2:.2f}")
print(f"\nVariancia: {variancia:.2f}")
```

Esperanca: 1.70

Esperanca^2: 3.70

Variancia: 0.81

Exercício 8

In [22]:

```
# Exercício 8

...

lambda = tx de ocorrencia
```



```

'''

def P_poisson(tx0correncia,k):
    if k < 0:
        return 0
    probabilidade = (math.exp(-tx0correncia) * (tx0correncia**k)) / math.factorial(k)
    return probabilidade

def varPoisson(tx0correncia):
    return tx0correncia
def espPoisson(tx0correncia):
    return tx0correncia

lam = 2    #por hora

print(f"a) Probabililidade de receber 3 chamadas em uma hora: {P_poisson(lam,3)}\n")
print(f"b) Probabilidade de receber nenhuma chamada: {P_poisson(lam,0)}")
print(f"c) Número esperado de chamdas em 3 horas: {espPoisson(3*lam)}")

```

a) Probabililidade de receber 3 chamadas em uma hora: 0.1804470443154836

b) Probabilidade de receber nenhuma chamada: 0.1353352832366127

c) Número esperado de chamdas em 3 horas: 6

Exercício 9

In [23]:

```

p = 0.1
q = 0.9

x = 10 #quantidade de casos avaliados

x_valores = np.arange(1, x + 1) #distribuição dos primeiro x casos
probabilidades = []

for i in x_valores:
    probabilidades.append((q**(i-1)) * p)

print("Distribuição:")
for x, pr in zip(x_valores, probabilidades):
    print(f"X = {x}: {pr:.4f}")

print(f"\nProbabilidade X=3: {probabilidades[2]*100:.2f}%")

esperanca = 1 / p
print(f"\nEsperança E(X): {esperanca:.1f} inspeções")

```

Distribuição:

```

X = 1: 0.1000
X = 2: 0.0900
X = 3: 0.0810
X = 4: 0.0729
X = 5: 0.0656
X = 6: 0.0590
X = 7: 0.0531
X = 8: 0.0478
X = 9: 0.0430

```

$X = 10: 0.0387$

Probabilidade $X=3$: 8.10%

Esperança $E(X)$: 10.0 inspeções

Exercício 10

In [24]:

```
# Exercício 10

import numpy as np
import matplotlib.pyplot as plt

x = [0 , 1 , 2 , 3 ]
p_x = [0.1 , 0.3, 0.4, 0.2]

# (a) Função de distribuição acumulada (CDF)
F_x = np.cumsum(p_x)

print("Valores de x:", x)
print("p(x):", p_x)
print("F(x):", F_x)

# (b) Cálculos pedidos
# i) F(1)
F_1 = F_x[1]

# ii) F(2)
F_2 = F_x[2]

# iii)  $P(1 < X \leq 3)$ 
P_intervalo = np.sum(p_x[2:])

print("\nResultados:")
print("F(1) =", F_1)
print("F(2) =", F_2)
print("P(1 < X ≤ 3) =", P_intervalo)

# Plot da CDF
plt.plot(x , F_x , 'bo', markersize =5) # pontos azuis
# Plota as linhas horizontais entre os pontos
for i in range(1 , len( x )):
    plt.hlines(F_x [i -1] , x [i -1] , x [i] , colors ='b', linestyle ='solid')

for i in range(1 , len( x ) - 1) :
    plt.vlines (x [i] , F_x [i -1] , F_x [i] , colors ='b', linestyle ='dotted')

plt.title (" Função de Distribuição Acumulada de X")
plt.xlabel ("x")
plt.ylabel (r" $F_X (x)$")
plt.grid ( True )
plt.show ()
```

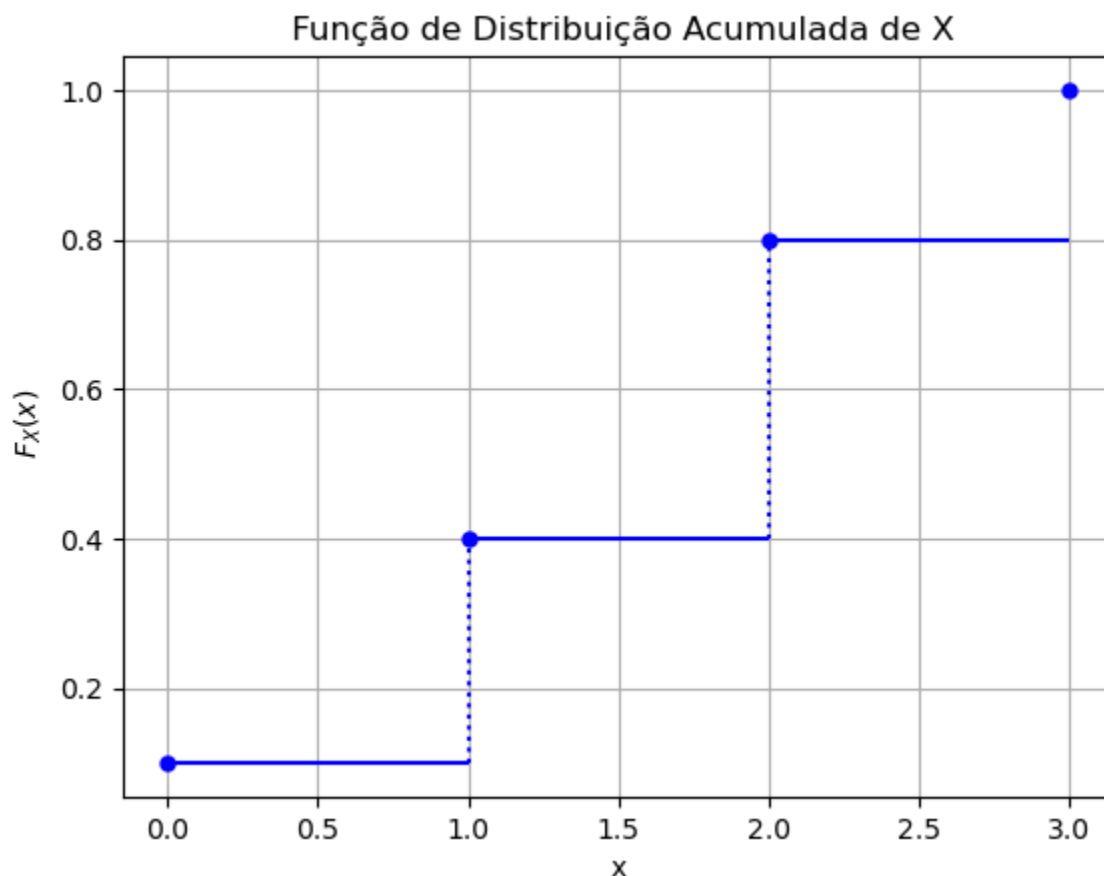
Valores de x: [0, 1, 2, 3]
p(x): [0.1, 0.3, 0.4, 0.2]
F(x): [0.1 0.4 0.8 1.]

Resultados:

$F(1) = 0.4$

$F(2) = 0.8$

$P(1 < X \leq 3) = 0.6000000000000001$



Exercício 11

In [25]:

```
p = 0.1
q = 1 - p

x = np.array([0, 1])
probabilidades = np.array([q, p])

esperanca = np.average(x, weights=probabilidades)
variancia = np.average((x - esperanca)**2, weights=probabilidades)

print(f"Distribuição de Bernoulli com p = {p}")
print(f"Esperanca: {esperanca:.2f}")
print(f"Variancia: {variancia:.2f}")
```

Distribuição de Bernoulli com $p = 0.1$

Esperanca: 0.10

Variancia: 0.09

Exercício 12

a) Número sorteado em um dado honesto.

Neste caso podemos usar a distribuição Uniforme Discreta, pois em um dados comum, todos os lançamentos tem resultados com a mesma oportunidade de ocorrerem($1/6$).

b) Resultado de inspeção de uma única peça (defeituosa ou não).

Aqui podemos usar a distribuição uniforme de Bernoulli, pois é um experimento com apenas dois resultados possíveis.

c) Número de caras em 10 lançamentos de moeda.

Neste caso usamos a Distribuição Binomial, visto que é a de Bernoulli aplicada n vezes

d) Número de chamadas recebidas em uma hora.

Neste caso usamos Poisson, pois modela o numero de ocorrencias de um evento em um intervalo de tempo não havendo limite máximo como na binomial e sim uma taxa média.